

Week2_예습과제_김재희

[통계](#) [수정](#) [삭제](#)

카사바 · 방금 전

 0

Euron_2026-2



▼ 목록 보기

1/1



순환 신경망

순환 신경망 모델: 순서가 있는 연속적인 데이터를 처리하는 데 적합한 구조를 갖고 있다.

연속형 데이터: 특정 시점 t 에서의 데이터가 이전 시점($t_0, t_1, \dots, t_{n-1}$)의 영향을 받는 데이터

순환 신경망

순환 신경망은 연속형 데이터를 순서대로 입력받아 처리하며 각 시점마다 은닉 상태(hidden state)의 형태로 저장한다. 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드를 순환 신경망의 셀이라고 한다.

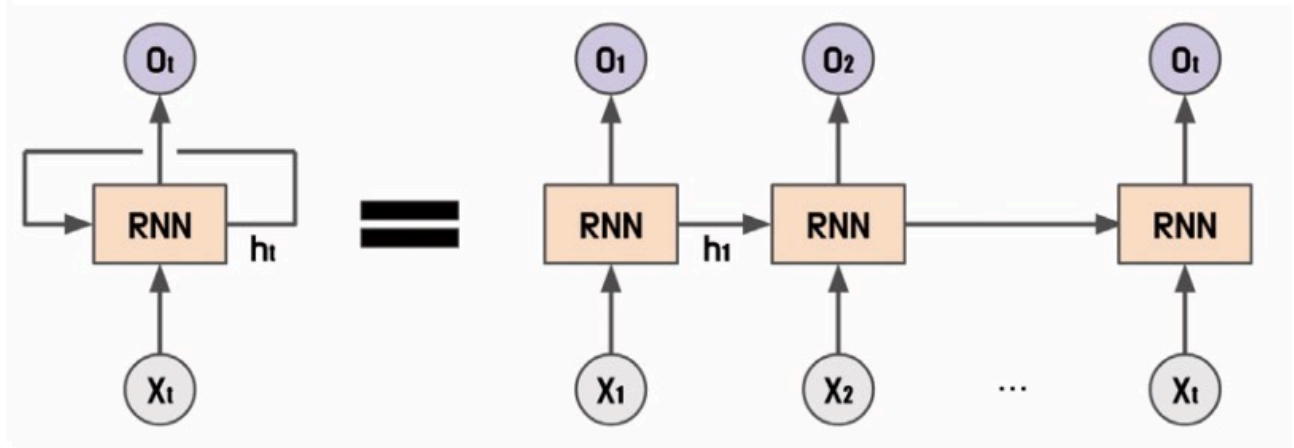


그림 6.12 순환 신경망의 셀

각 시점 t 에서 현재 입력값 x_t 와 이전 시점 $t-1$ 의 은닉 상태 h_{t-1} 를 사용해 현재 시점의 은닉 상태 h_t 와 출력값 y_t 를 계산한다.

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

일대다 구조

일대다 구조: 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 구조
e.g. 문장에서 각 단어의 품사를 예측하는 작업, 이미지 캡셔닝

다대일 구조

다대일 구조: 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 구조
e.g. 감성 분류, 문장 분류, 자연어 추론

다대다 구조

다대다 구조: 입력 시퀀스와 출력 시퀀스 길이가 여러 개인 경우에 사용되는 구조
e.g. 번역기, 음성 인식기

양방향 순환 신경망

양방향 순환 신경망: 기본 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식. 이전 시점($t-1$)의 은닉 상태뿐만 아니라 이후 시점($t+1$)의 은닉 상태도 함께 이용한다.

다중 순환 신경망

다중 순환 신경망: 여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환 신경망이 서로 다른 정보를 처리하도록 설계되어 있다. 이렇게 여러 개의 층으로 구성된 RNN은 데이터의 다양한 특징을 추출할 수 있어 성능이 향상될 수 있다. 또, 층이 깊어질수록 더 복잡한 패턴을 학습할 수 있다는 장점이 있다. 하지만 RNN 층이 많아질수록 학습 시간이 오래 걸리고, 기울기 소실 문제가 발생할 가능성도 높아진다.

장단기 메모리

장단기 메모리: 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 모델

일반적인 순환 신경망은 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값을 예측하므로 먼 과거의 정보를 잘 기억하지 못한다. 이 때문에 장기 의존성 문제가 발생할 수 있다. 또한, 활성화 함수로 사용되는 하이퍼볼릭 탄젠트 함수나 ReLU 함수의 특성으로 인해 역전파 과정에서 기울기 소실이나 폭주가 발생할 가능성이 있다.

이러한 문제를 해결하기 위해 장단기 메모리를 사용한다. 장단기 메모리는 메모리 셀과 게이트라는 구조를 도입해 장기 의존성 문제와 기울기 소실 문제를 해결한다.

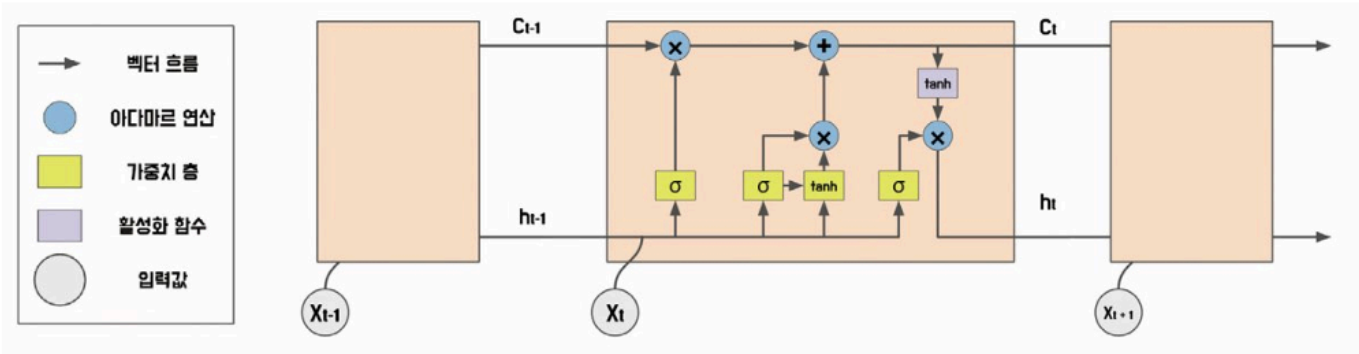


그림 6.18 장단기 메모리 구조

셀 상태는 정보를 저장하고 유지하는 메모리 역할을 하며 출력 게이트와 망각 게이트에 의해 제어된다.

망각 게이트: 장단기 메모리에서 이전 셀 상태에서 어떠한 정보를 삭제할지 결정하는 역할을 하며 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 삭제할지 결정한다.

입력 게이트: 새로운 정보를 어떤 부분에 추가할지 결정하는 역할을 하며 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 추가할지 결정한다.

출력 게이트: 셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할을 하며 현재 입력과 이전 셀 상태, 새로 추가된 정보를 입력으로 받아 출력할 정보를 결정한다.

장단기 메모리 구조

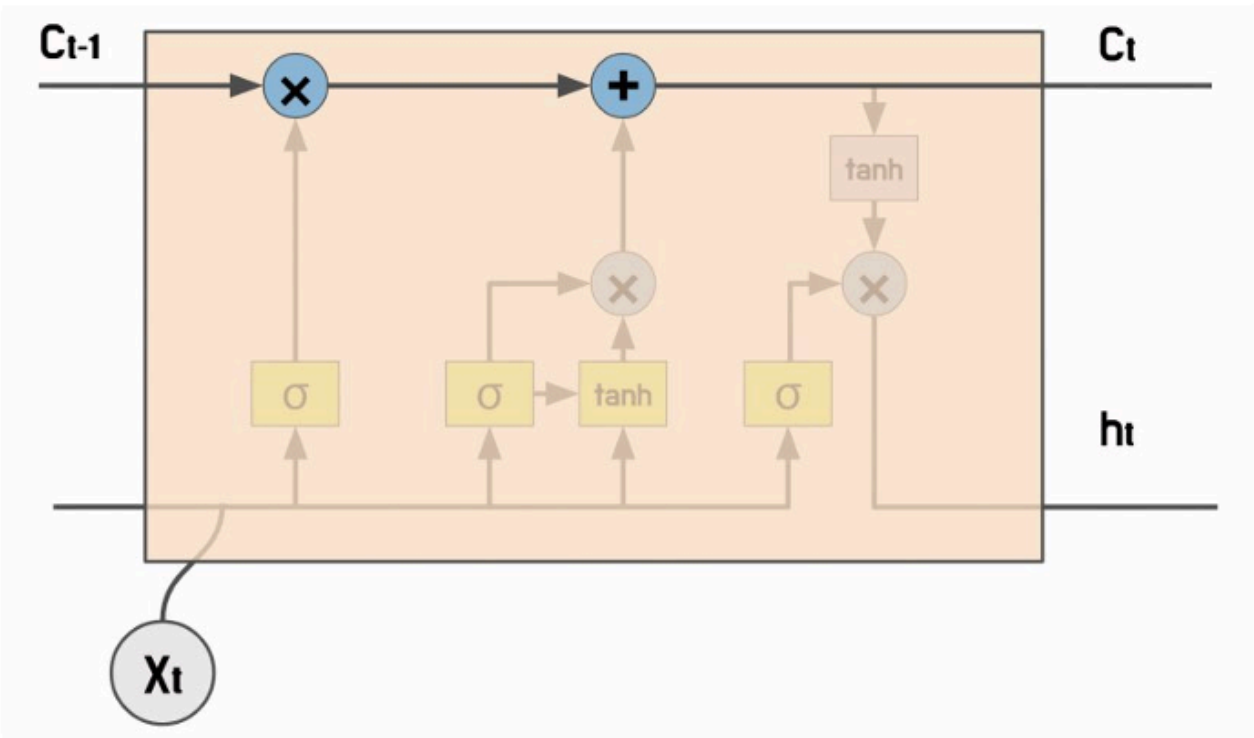


그림 6.19 장단기 메모리 셀 상태

메모리 셀은 순환 신경망의 은닉 상태와 유사하게 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장하는 역할을 한다. 하지만 순환 신경망에서 은닉 상태는 출력값을 계산하는 데 사용되지만, 장단기 메모리의 메모리 셀은 출력값 계산에 직접 사용되지 않는다. 대신 장단기 메모리는 게이트들을 통해 어떤 정보를 버릴지, 어떤 정보를 기억할지, 어떤 정보를 출력할지를 결정하는 추가적인 연산을 수행한다.

세 가지 게이트 모두 활성화 함수로 시그모이드를 사용한다. 출력값은 0에서 1사이 값인데, 이 값이 해당 게이트가 입력값에 대해 얼마나 많은 정보를 통과시킬지 결정한다. 예를 들어 망각

게이트 출력값이 1이면 이전 시점 기억 상태가 현재 시점 기억 상태에 완전히 유지되며, 0이면 이전 시점 기억 상태는 전혀 반영되지 않는다.

망각 게이트: 현재 시점 입력과 이전 시점 은닉 상태를 입력으로 받아 시그모이드 함수를 거친 값과 메모리 셀을 곱한 값으로 이전 시점의 메모리 셀을 업데이트한다.

기억 게이트: 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수를 거친 값과 하이퍼볼릭 탄젠트 함수를 거친 값의 곱으로 새로운 기억 값을 계산한다. 기억 게이트는 어떤 정보를 얼마나 추가할지를 결정한다.

출력 게이트: 현재 시점의 입력과 이전 시점의 은닉 상태, 새로 업데이트된 메모리 셀을 입력으로 받아 현재 시점의 출력값을 계산한다. 출력 게이트는 어떤 정보를 출력할지를 결정한다.

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

수식에서 W 는 입력값과 은닉 상태 가중치를 의미하며, b 는 망각 게이트의 편향을 의미한다.

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$
$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

기억 게이트는 g_i 와 i_t 로 구성돼 있으며 망각 게이트와 동일한 연산으로 계산된다. g_i 는 활성화 함수로 하이퍼볼릭 탄젠트를 사용하며 i_t 는 시그모이드를 사용한다.

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

f_t 는 망각 게이트 출력값이며 c_{t-1} 은 이전 시점의 메모리 셀 값을 나타낸다. 이 값을 원소별 곱셈 연산하면 현재 시점의 메모리 셀 값이 계산된다.

따라서 망각 게이트 값이 0에 가까울수록 이전 시점 메모리 셀 값이 현재 시점 메모리 셀 값에 영향을 미치지 않게 된다.

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

출력 게이트는 현재 시점의 은닉 상태를 제어한다.

$$h_t = o_t \odot \tanh(c_t)$$

현재 시점의 은닉 상태 h_t 는 출력 게이트와 하이퍼볼릭 탄젠트를 적용한 메모리 셀 값으로 계산된다.

Transformer

트랜스포머: 딥러닝 모델 중 하나로 다양한 자연어 처리 분야에서 많은 성과를 내는 모델이다. 어텐션 메커니즘만을 사용해 시퀀스 임베딩을 표현한다.

인코더는 입력 시퀀스를 임베딩하여 고차원 벡터로 변환하고, 디코더는 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성한다. 이때 어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중한다. 이를 통해 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악하여 번역이나 요약문 생성과 관련된 작업 등을 수행할 수 있게 된다.

트랜스포머는 기존 RNN보다 학습 속도가 빠르고 병렬 처리가 가능해 대규모 데이터셋에서 높은 성능을 보인다. 또 임베딩 과정에서 문장 전체 정보를 고려하기 때문에 문장의 길이가 길어지더라도 성능이 유지된다.

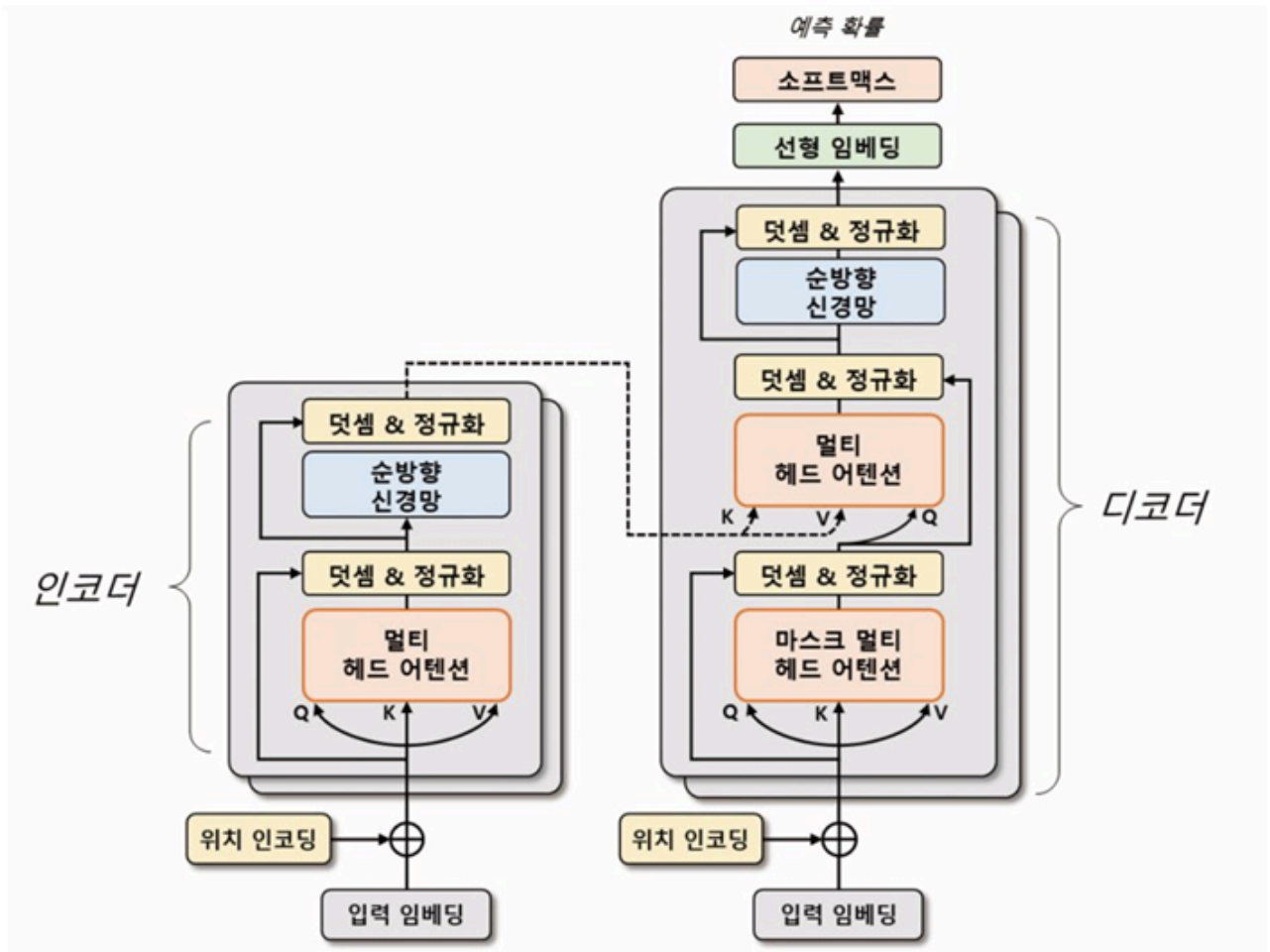


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

트랜스포머는 인코더와 디코더 두 부분으로 구성돼 있으며, 각각 N개의 트랜스포머 블록으로 구성된다. 이 블록은 멀티 헤드 어텐션과 순방향 신경망으로 이뤄져 있다.

멀티 헤드 어텐션: 입력 시퀀스에서 Q, K, V 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법. 이 과정에서 쿼리와 각 키의 유사도를 계산하고, 해당 유사도를 가중치로 사용하여 값 벡터를 합산한다.

이렇게 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체한다. 결국 입력 시퀀스 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신한다.

순방향 신경망은 이 과정에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용된다. 이 신경망은 여러 개의 선형 계층으로 구성돼 있으며, 앞선 순방향 신경망의 구조와 동일하게 입력 벡터에 가중치를 곱하고 편향을 더하며 활성화 함수를 적용한다. 이 과정에서 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신된다.

인코더는 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현해 트랜스포머 블록의 출력 벡터를 생성한다.

디코더는 마스크 멀티 헤드 어텐션을 사용해 타깃 시퀀스 데이터를 순차적으로 생성한다. 이때 디코더 출력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조한다.

입력 임베딩과 위치 인코딩

입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환된다. 트랜스포머 모델은 순환 신경망과 달리 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보를 제공하지 않는다.

따라서 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 한다. 이를 위해 위치 인코딩 방식을 사용한다.

위치 인코딩 방식은 각 단어의 위치 정보를 나타내는 벡터를 더해 임베딩 벡터에 위치 정보를 반영한다. 위치 인코딩 벡터는 $\sin$ 함수와 $\cos$ 함수를 사용해 생성되며 이를 통해 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성한다.

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

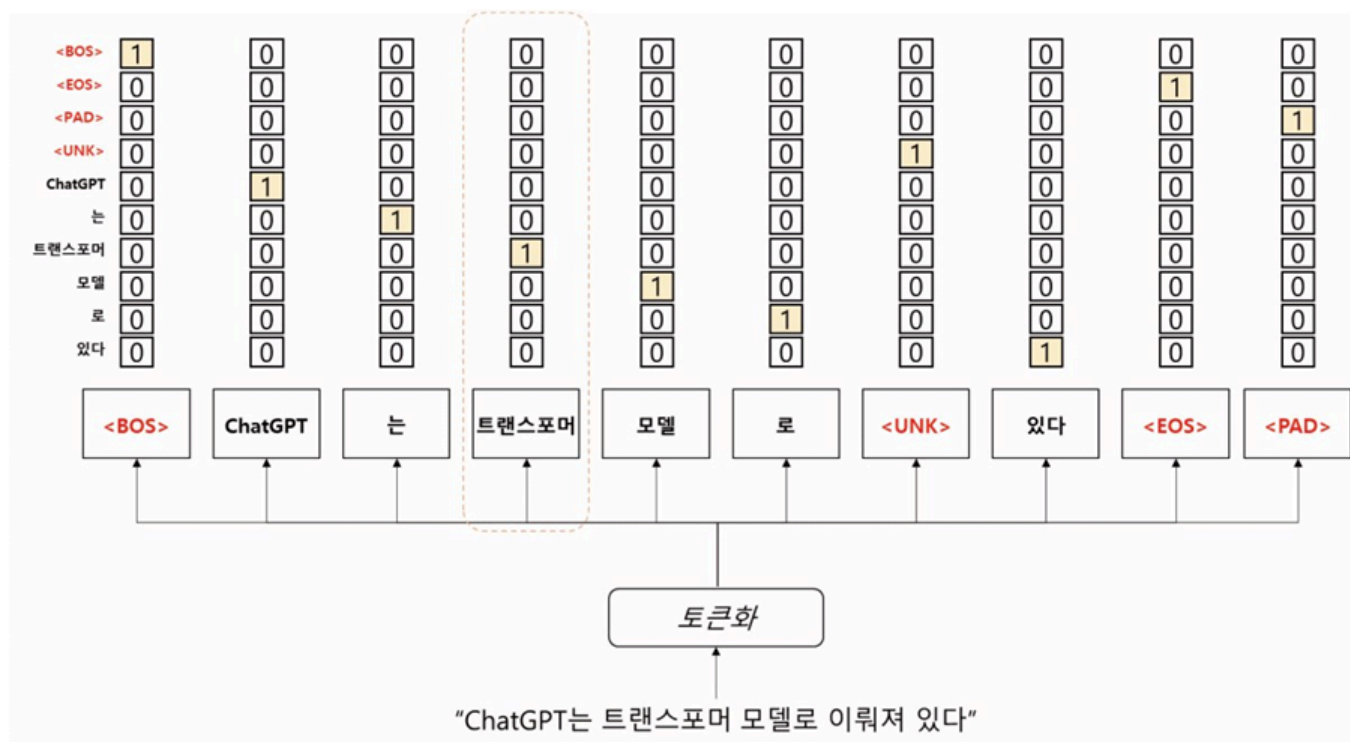
pos 는 입력 시퀀스에서 해당 단어의 위치를 나타낸다.

i 는 임베딩 벡터의 차원 인덱스를 의미한다.

차원 인덱스가 짝수라면 첫 번째 $\sin$ 함수 수식을 적용하며 홀수라면 두 번째 $\cos$ 함수 수식을 적용한다.

특수 토큰

트랜스포머는 단어 토큰 이외에 특수 토큰을 활용해 문장을 표현한다. 이 특수 토큰은 입력 시퀀스의 시작과 끝을 나타내거나 마스킹 영역으로 사용된다.



BOS: 문장의 시작

EOS: 문장의 종료

UNK: 어휘 사전에 없는 단어로 모르는 단어를 의미한다.

PAD: 모든 문장을 일정한 길이로 맞추기 위해 사용된다. 짧은 문장의 빈 공간을 채워준다.

이렇게 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현한다.

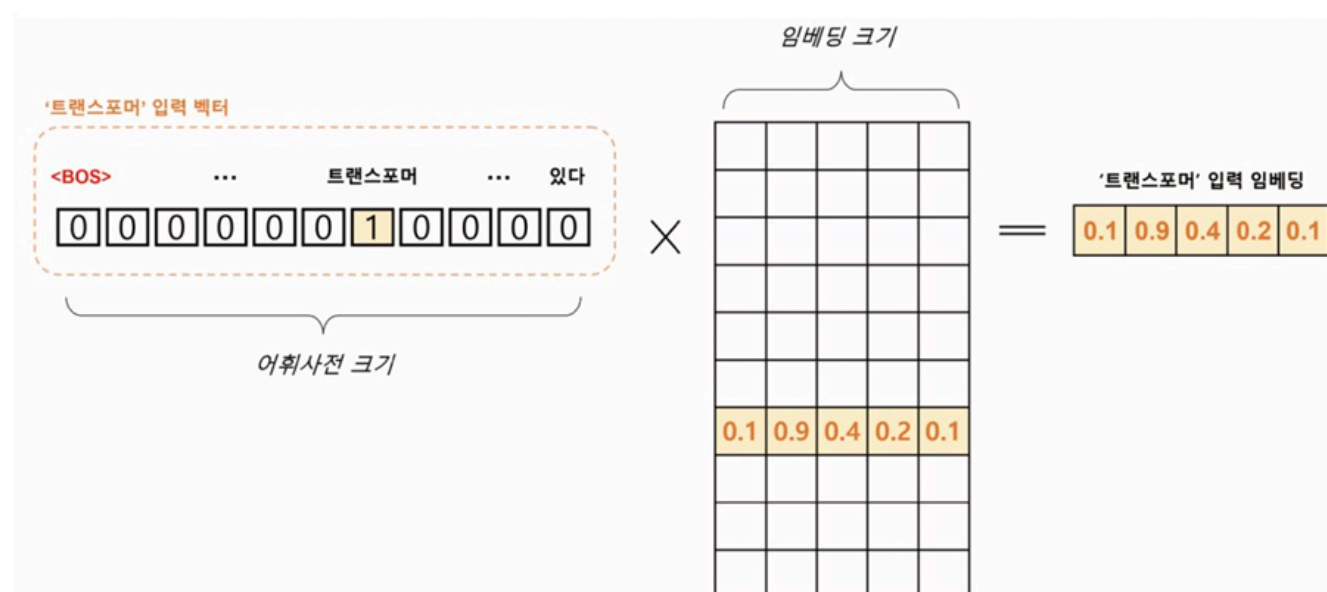


그림 7.4 입력 임베딩 산출 과정

어휘 사전 크기를 V , 입력 임베딩 차원을 d 라고 했을 때, '트랜스포머'의 원-핫 벡터는 $[1, v]$ 크기를 갖는다. 이 원-핫 벡터는 임베딩 행렬 $[v, d]$ 에 의해 $[1, d]$ 크기 벡터로 변환된다.

트랜스포머 인코더

인코더는 입력 시퀀스를 받아 여러 계층으로 구성된 인코더 계층을 거쳐 연산을 수행한다. 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성되며 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달한다. 산출된 인코더 계층의 출력은 디코더 계층으로 전달된다.

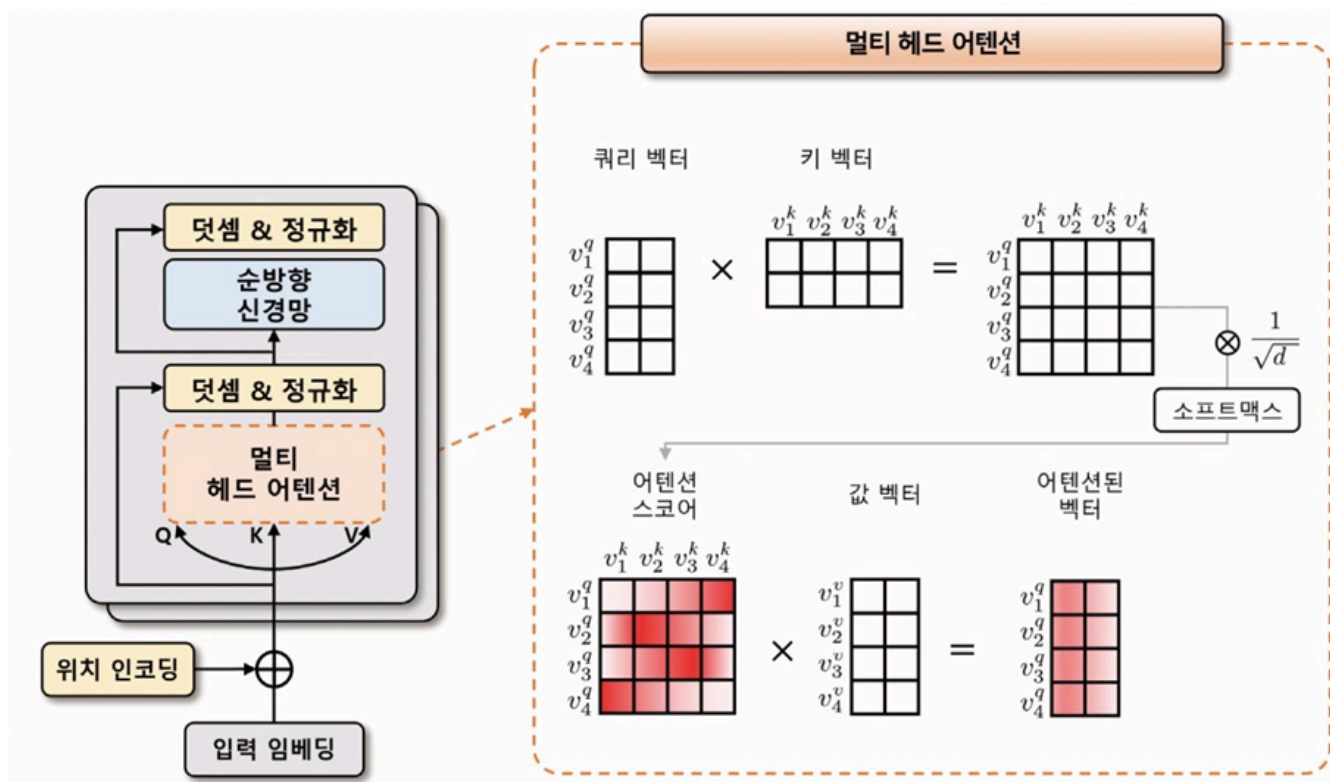


그림 7.5 트랜스포머 인코더 블록 연산 과정

쿼리 벡터: 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터로 인코더의 각 시점마다 생성된다. 현재 시점에서 질문이 되는 벡터를 의미하며 이 벡터를 기준으로 다른 시점의 정보를 참조한다.

키 벡터: 쿼리 벡터와 비교되는 대상으로 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터가 된다.

값 벡터: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할을 한다.

쿼리와 키 벡터의 연관성은 내적 연산으로 $[N, s, s]$ 어텐션 스코어 맵을 사용한다.

수식 7.2 어텐션 스코어 계산식

$$\text{score}(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

멀티 헤드: 셀프 어텐션을 여러 번 수행해 여러 개의 헤드를 만든다. 각각의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합친다.

순방향 신경망은 두 가지 방법을 적용할 수 있다. 선형 임베딩과 ReLU로 이뤄진 인공 신경망이나 1차원 합성곱이 사용된다. 이어서 다시 덧셈, 정규화 과정이 수행된다.

트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성된다. 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용되며 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용된다.

트랜스포머 디코더

트랜스포머 디코더는 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력받는다.

트랜스포머 모델에서 인코더의 멀티 헤드 어텐션 모듈은 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체된다.

어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 볼 수 있게 마스크를 씌운다. 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌운다. 이렇게 마스크를 적용하면 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며 인과성이 보장된다. 마스크 멀티 헤드 어텐션 모듈에서는 마스크 영역에 수치적으로 굉장히 작은 값인 $-\infty$ 마스크를 더해줌으로써 해당 영역의 어텐션 스코어값이 0에 가까워진다. 소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0이므로 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 된다.

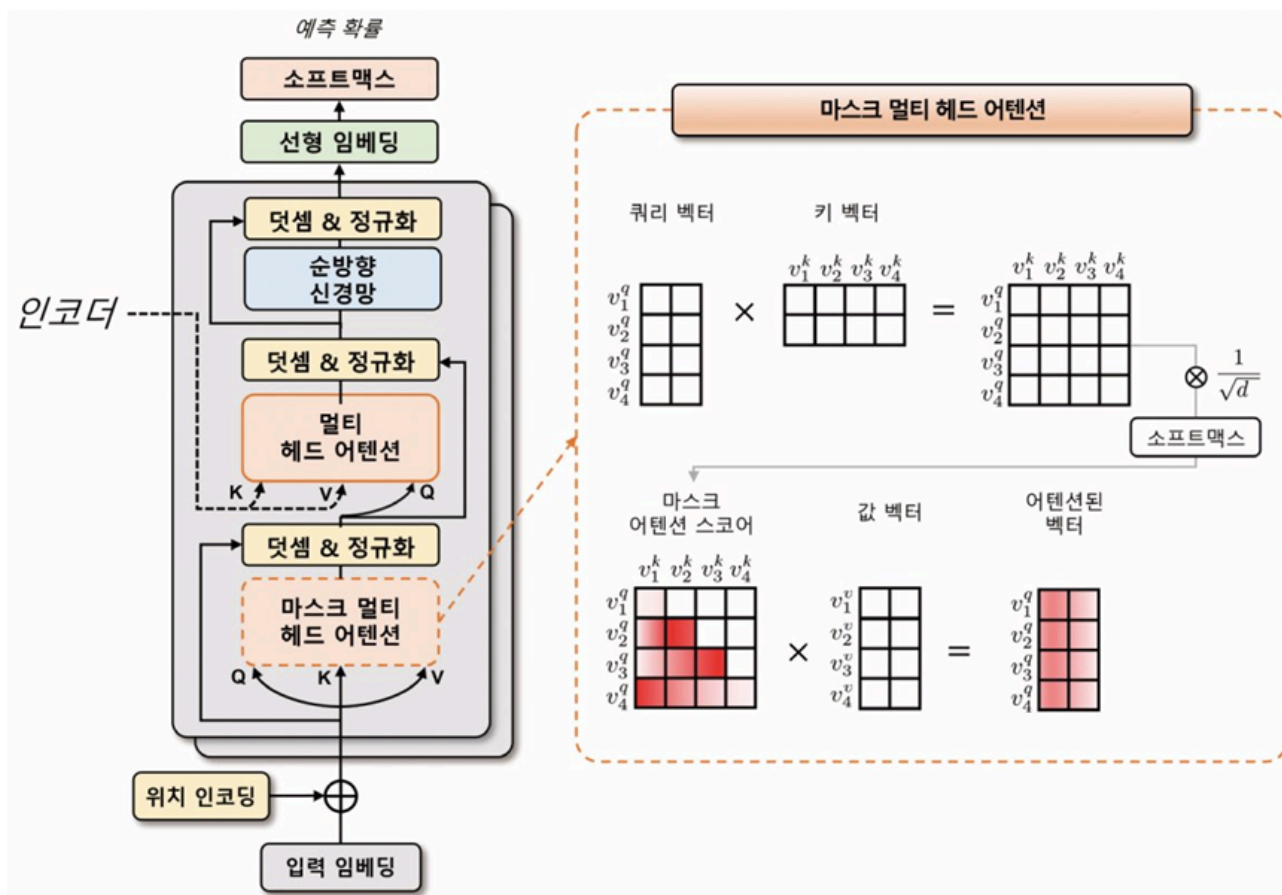
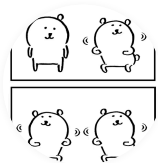


그림 7.6 트랜스포머 디코더 블록 연산 과정

디코더의 멀티 헤드 어텐션에서는 타깃 데이터가 쿼리 벡터로 사용되며 인코더의 소스 데이터가 키와 값 벡터로 사용된다.

이후 쿼리 벡터, 키 벡터, 값 벡터를 이용해 어텐션 스코어 맵을 계산하고 소프트맥스 함수를 적용하여 어텐션 가중치를 구한다. 최종으로 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻을 수 있다.



카사바

안녕하세요 :)

0개의 댓글

댓글을 작성하세요

댓글 작성

